

Lab 13 · La oficina en una caja

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

60 minutos · Spring Boot 4.1.0 · Java 25 (Temurin)

Resumen

Que la aplicación se empaqueta entera —con su servidor dentro— en **un solo archivo de 21 MB** que arranca con `java -jar`, partido en cuatro capas por velocidad de cambio. Y que **el mismo artefacto** se comporta distinto según dónde arranque, sin reconstruir nada.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	3
2. El caso	3
2.1. La mudanza, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · El jar	3
3.2. Paso 2 · Las capas	4
3.3. Paso 3 · Qué es un contenedor	6
3.4. Paso 4 · La misma caja, dos comportamientos	7
4. Lo que aprendiste	9
5. Lo que no vimos hoy	10
6. Para profundizar	10
7. Antes de cerrar	10

1 Antes de empezar

1.1 Qué vas a lograr

Tu aplicación funciona **en tu máquina**. Hoy la vas a meter en una caja que funciona en cualquier otra.

Vas a empaquetarla en un jar, vas a ver por qué ese jar viene **partido en capas**, vas a construir una **imagen de contenedor sin tener Docker**, vas a abrirla con `tar` para ver qué hay dentro, y vas a comprobar que el mismo artefacto se comporta de dos maneras distintas según una variable de entorno.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs anteriores	Sabes arrancar y configurar	—
Estar en la carpeta del lab	<code>cd labs/lab-13-</code> <code>empaquetado/practica</code>	El <code>cd</code> no da error
NO hace falta Docker	—	Y es el punto del laboratorio

Aquí no se usa `spring-boot:run`. Hoy se construye y se arranca el artefacto: `./mvnw package y java -jar`. Es más parecido a lo que pasa en un servidor.

Y para `java -jar` hay que usar el Java del repositorio, no el que tengas instalado. En el paso 1 está el comando y el error que sale si te equivocas.

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. El XML del `pom.xml` lo aguanta —la sangría ahí es adorno—, pero el **YAML no**: los dos archivos de perfil del paso 5 están marcados como «se escribe», y se teclean. El código completo está en `labs/lab-13-empaquetado/solucion/`.

2 El caso

La DGT tiene que llevar su oficina a otro edificio. No basta con llevar los papeles: hay que llevar **los muebles, la instalación eléctrica y el manual de apertura**, y que al llegar todo funcione igual.

2.1 La mudanza, que es la metáfora de este laboratorio

Meter la oficina entera en una caja, y que abra igual en cualquier sitio.

Dentro de la caja va todo lo que la oficina necesita para funcionar: el mobiliario (tu código) y las herramientas (las bibliotecas, **incluido el servidor web**). Nadie en el destino tiene que instalar un servidor de aplicaciones ni saber qué versión hace falta.

Y la caja se hace **por capas**, como se embala de verdad:

- Abajo, lo pesado que **nunca cambia**: las estanterías, los archivadores. Son las dependencias.
- Arriba, lo ligero que **cambia cada día**: los papeles de encima de la mesa. Es tu código.

Así, cuando cambias una línea de código, no vuelves a embalar los archivadores: **cambias la caja de arriba**. Kilobytes en vez de ciento treinta y ocho megas — que es lo que pesa la mudanza completa el día que esta caja acabe dentro de un contenedor.

Y una cosa que NO va dentro de la caja: el cartel de la puerta. El cartel dice si esto es la oficina de pruebas o la de verdad, y a qué Tesorería llama. Eso se pone **al llegar**, en el destino. Si viajara dentro, tendrías que hacer una caja distinta por cada edificio — y entonces la que probaste no sería la que abriste.

3 Los pasos

3.1 Paso 1 · El jar

Qué vamos a hacer

Empaquetar la aplicación en un solo archivo y arrancarla sin Maven.

Para entenderlo mejor

Cerrar la caja. A partir de aquí, lo que se mueve es **un archivo**.

El problema

`./mvnw spring-boot:run` compila, resuelve dependencias y arranca — y para eso necesita Maven, el código fuente y el repositorio de dependencias. En un servidor no hay nada de eso.

La alternativa, y por qué no

- **Un .war desplegado en un Tomcat instalado aparte:** es como se hacía, y significa que tu aplicación **no se puede arrancar sola**. Depende de una máquina que preparó otra persona, y lo que pruebas no es lo que corre.
- **Un jar ejecutable con el servidor dentro,** que es lo de aquí: `java -jar` y ya está. Lo mismo en tu portátil y en producción.

Se corre

```
./mvnw package
```

Y para arrancarlo, **con el Java del repositorio:**

```
../../../../tools/jdk/runtime/macos-aarch64/jdk-25.0.4+7/Contents/Home/bin/java \
-jar target/lab13-empaquetado-0.1.0.jar
```

Lo que vas a ver

```
target/lab13-empaquetado-0.1.0.jar    20.9 MB
```

Veinte megas: tu código es una parte mínima; el resto son Spring, Tomcat y las bibliotecas.

Vas bien si...

El package termina en `BUILD SUCCESS` y en `target/` hay un `.jar` de unos 20 MB.

Si te atascas

1 · `UnsupportedClassVersionError ... class file version 69.0 ... up to 65.0`

Exception in thread "main" java.lang.UnsupportedClassVersionError:
cl/dgt/empaquetado/Lab13Application has been compiled by a more recent version of
↪ the
Java Runtime (class file version 69.0), this version of the Java Runtime only
recognizes class file versions up to 65.0

Estás arrancando con el Java de tu máquina, no con el del repositorio. El jar está compilado con Java 25 (versión de clase 69) y tu java es más viejo — 65 es Java 21.

Comprueba cuál tienes con `java -version`, y usa la ruta completa al Java del repositorio, como en el comando de arriba. `./mvnw` lo hace solo; `java -jar` no.

2 · `no main manifest attribute`

Empaquetaste sin el plugin de Spring Boot, así que el jar no sabe arrancarse. Comprueba que el `spring-boot-maven-plugin` está en el `pom.xml`.

3.2 Paso 2 · Las capas

Qué vamos a hacer

Partir el jar por velocidad de cambio, y mirar las cuatro capas.

Para entenderlo mejor

Embalar por separado lo que nunca cambia y lo que cambia a diario.

El problema

Un jar es **una sola pieza**. Si cambias una línea de código, la pieza entera es distinta — y el día que ese jar viaje dentro de una imagen de contenedor, eso significa volver a subir y bajar los 138 megas enteros por una letra.

La alternativa, y por qué no

- **Jar plano**: lo de fábrica, y perfecto cuando sólo copias el jar a un servidor. Ahí las capas no aportan nada.
- **Jar por capas**, que es lo de aquí: sólo tiene sentido cuando hay una imagen de por medio — y entonces cambia mucho, porque lo pesado deja de viajar. Hoy no vas a construir esa imagen (ver el paso 3), pero el jar queda listo para el día que alguien lo haga.

Se pega

En practica/pom.xml, **dentro del plugin de Spring Boot**:

```
<configuration>
  <layers>
    <enabled>true</enabled>
  </layers>
</configuration>
```

Se corre

```
./mvnw package
<el java del repositorio> -Djarmode=tools -jar target/lab13-empaquetado-0.1.0.jar
↪ list-layers
```

Lo que vas a ver

```
dependencies
spring-boot-loader
snapshot-dependencies
application
```

Cuatro capas, ordenadas de menos a más volátil. `dependencies` es casi todo el peso y cambia cuando cambias una versión —una vez cada varios meses—. `application` es tu código y cambia cada vez que guardas.

Vas bien si...

El comando lista las cuatro capas, con `application` la última.

Si te atascas

1 · Unknown jar mode

No activaste las capas en el `pom.xml`, o no volviste a empaquetar después de activarlas.

3.3 Paso 3 · Qué es un contenedor

Qué vamos a hacer

Nada. Veinte minutos de explicación, y es donde el paso 2 cobra sentido.

El problema

Despliegas el jar del paso 1 en un servidor y no arranca: el servidor tiene Java 17 y tu jar se compiló con Java 25. Instalas Java 25 y rompes **otra** aplicación que necesitaba la 17. Y aparece la frase:

«En mi máquina funciona.»

Y es literalmente cierta. El jar es idéntico; lo que cambia es **todo lo que hay debajo**: la versión de Java, las bibliotecas del sistema, la zona horaria, la codificación por defecto.

La alternativa, y por qué no

- **Instalar a mano lo que haga falta en cada servidor**: es lo que se hacía, y es de donde viene el problema.
- **Una máquina virtual**: llevarse el sistema operativo entero en un archivo. Funciona, y cuesta varios GB y un minuto de arranque por cada copia.
- **Un contenedor**, que es lo de aquí: **comparte el núcleo del sistema y aísla todo lo demás**.

Qué es un contenedor, entonces

No lleva sistema operativo. Es un **proceso normal** de la máquina anfitriona al que el núcleo le ha mentado sobre el mundo: ve su propio sistema de archivos, sus propios procesos —se cree el número 1—, su propia red con su propia IP, y tiene un tope de memoria y de procesador. Lo hace el núcleo de Linux con dos mecanismos: *namespaces* (qué ve) y *cgroups* (cuánto consume).

	máquina virtual	contenedor
lleva sistema operativo	sí , completo	no: comparte el núcleo
tamaño típico	varios GB	decenas o cientos de MB
arranque	de 30 s a minutos	milisegundos
aislamiento	total (hardware virtual)	bueno, pero comparte núcleo

La **imagen** es la plantilla; el **contenedor** es una imagen corriendo. Misma relación que entre una clase y un objeto.

Y aquí es donde cobra el paso 2

La imagen está hecha de **capas apiladas**, de sólo lectura y **compartidas**: si diez imágenes usan la misma base de Java, esa base se guarda una vez. Una imagen de esta aplicación repartiría así sus ciento treinta y ocho megas:

~120 MB	la base: sistema mínimo + JRE 25	} no cambian casi nunca } cambia cada día
~18 MB	las dependencias (Spring, Tomcat...)	
< 0,1 MB	TU CÓDIGO	

Las cuatro capas del jar se convierten en capas de la imagen. Y entonces:

Al desplegar una corrección no se mueven 138 MB. Se mueve **la capa que cambió**, unos kilobytes. Todo lo demás ya está en el servidor.

Sin el `layers.enabled` del paso 2, el jar sería un solo archivo indivisible dentro de la imagen y cualquier cambio movería los 21 MB enteros.

OCI, Docker, y por qué hoy no construyes ninguna imagen

OCI (*Open Container Initiative*) es el **estándar** de formato de imagen. **Docker** es un programa que construye y ejecuta imágenes OCI: el más conocido, y no el único.

Y la parte honesta: **en las máquinas de esta sala no hay Docker y no se puede instalar**. Así que este laboratorio explica qué es un contenedor y no construye ninguno. Fabricar un archivo que nadie puede ejecutar, para abrirlo con `tar` y mirar un JSON, ocupa media hora y enseña menos que este dibujo. El nombre de la herramienta que sí lo haría está en «lo que no vimos hoy».

Vas bien si...

Puedes explicar con tus palabras en qué se diferencia un contenedor de una máquina virtual, y por qué el jar por capas del paso 2 hace más barato un despliegue.

3.4 Paso 4 · La misma caja, dos comportamientos

Qué vamos a hacer

Arrancar **el mismo jar** tres veces y obtener tres comportamientos, sin recompilar nada.

Para entenderlo mejor

El cartel de la puerta. La caja es la misma; lo que cambia es lo que pones al llegar.

El problema

Si la configuración viaja dentro del artefacto, hace falta **un artefacto por entorno**. Y entonces el que probaste no es el que desplegaste — que es exactamente la clase de diferencia que produce los incidentes que nadie sabe reproducir.

La alternativa, y por qué no

- **Un artefacto por entorno**: se hace, y sólo tiene sentido cuando cambian las dependencias y no la configuración.
- **Perfiles + variables de entorno**, que es lo de aquí: un artefacto, y el entorno decide. Y hay un orden de precedencia: **la variable de entorno gana al perfil, y el perfil gana al `application.yml`**.

Se escribe — aquí no se pega nada

El YAML se escribe a mano: la sangría es el significado. Al copiarlo del PDF se pierden los espacios del principio de línea y el archivo deja de decir esto, **sin dar error**. Son **dos espacios por nivel**, y ninguna tabulación.

Archivo **nuevo** practica/src/main/resources/application-dev.yml — el archivo entero:

```
lab13:
  saludo: Hola desde DESARROLLO
  tesoreria-url: http://localhost:9098/pagos
```

Y practica/src/main/resources/application-prod.yml — entero:

```
lab13:
  saludo: Hola desde PRODUCCIÓN
  tesoreria-url: ${TESORERIA_URL:https://tesoreria.example.cl/pagos}
```

Lee despacio esa última línea, que es la que enseña la precedencia:

```
${NOMBRE_DE_VARIABLE:valor-de-respaldo}
```

Si existe la variable de entorno, gana ella. Si no, el texto de después de los dos puntos.

Y ese respaldo es un respaldo, no el valor de producción. `example.cl` no existe **a propósito**: si algún día esa URL aparece en un log de producción, es que nadie puso la variable — y se ve enseguida, en vez de apuntar en silencio a un sitio equivocado.

Se corre

```
SPRING_PROFILES_ACTIVE=dev <java> -jar target/lab13-empaquetado-0.1.0.jar
SPRING_PROFILES_ACTIVE=prod <java> -jar target/lab13-empaquetado-0.1.0.jar
SPRING_PROFILES_ACTIVE=prod TESORERIA_URL=https://tesoreria.sii.cl/api/pagos \
  <java> -jar target/lab13-empaquetado-0.1.0.jar
```

y en cada caso: `curl localhost:8105/donde-estoy`

Lo que vas a ver

```
dev:
{"saludo":"Hola desde DESARROLLO","tesoreriaUrl":"http://localhost:9098/pagos",
 "perfilesActivos":["dev"],"javaVersion":"25.0.4"}

prod, sin variable:
{"saludo":"Hola desde
↪ PRODUCCIÓN","tesoreriaUrl":"https://tesoreria.example.cl/pagos",
 "perfilesActivos":["prod"],"javaVersion":"25.0.4"}

prod, con TESORERIA_URL:
{"saludo":"Hola desde
↪ PRODUCCIÓN","tesoreriaUrl":"https://tesoreria.sii.cl/api/pagos",
 "perfilesActivos":["prod"],"javaVersion":"25.0.4"}
```


Mismo jar. Tres comportamientos. No se reconstruyó nada entre una ejecución y la siguiente: es el mismo archivo, byte por byte.

Y la regla que cierra el laboratorio: NINGÚN SECRETO en estos archivos.

Ni la contraseña de la base, ni el secreto de firma del JWT del Lab 09, ni una clave de API. Y la razón es concreta, no una buena práctica abstracta: **estos archivos viajan DENTRO del jar**, y un jar se descomprime con unzip como cualquier zip. El día que ese jar acabe dentro de una imagen, peor todavía: una imagen se sube a un registro y se abre con tar. Cualquiera que tenga el artefacto tiene estos archivos.

Además están en el repositorio, así que el secreto quedaría en el historial de Git **para siempre**, y borrarlo después no lo borra.

Los secretos van por **variable de entorno**, que las pone quien despliega al arrancar y no quedan escritas en ninguna parte.

Vas bien si...

Los tres arranques dan tres respuestas distintas, y en los tres el javaVersion es 25.0.4.

Si te atascas

1 · Los tres dicen lo mismo.

El perfil no está llegando. En Windows, `SPRING_PROFILES_ACTIVE=prod java -jar ...` **no funciona**: hay que usar `set` antes, o pasarlo como `--spring.profiles.active=prod`.

2 · Could not resolve placeholder 'TESORERIA_URL'

Escribiste `${TESORERIA_URL}` sin respaldo en un perfil que no lo define. Es el comportamiento correcto en producción para un secreto — pero para esta URL hay respaldo.

4 Lo que aprendiste

1 · Un jar ejecutable lleva el servidor dentro.

`java -jar` y ya está: lo mismo en tu portátil y en el servidor. No hay que instalar ni configurar un contenedor de aplicaciones aparte.

2 · Las capas existen para no mover lo que no cambió.

Dependencias abajo, tu código arriba. Cambiar una línea mueve kilobytes en vez de 138 MB — y sólo importa cuando hay una imagen de por medio.

3 · Un contenedor no es una máquina virtual.

No lleva sistema operativo: es un proceso al que el núcleo le aisló lo que ve. Por eso arranca en milisegundos y pesa cientos de MB en vez de gigabytes. Y una imagen no es magia: es un tar con capas y un JSON.

4 · El mismo artefacto, y el entorno decide.

Perfiles y variables de entorno, con una precedencia clara. Un artefacto por entorno rompe la única garantía que tienes: que lo que probaste es lo que desplegaste. Y ahí dentro **no van secretos**.

5 Lo que no vimos hoy

Construir la imagen. El paso 3 explicó qué es un contenedor y no construyó ninguno, porque en esta sala no hay Docker. Si quieres probarlo fuera del curso, el nombre es **Jib** (`jib-maven-plugin`, de Google): construye una imagen OCI **sin demonio Docker** —escribe el `.tar` directamente, sin permisos de administrador y sin `Dockerfile`—, y toma el jar por capas del paso 2 para convertir cada capa del jar en una capa de la imagen. Se declara en el `pom.xml`, se ejecuta con `./mvnw package jib:buildTar`, y el resultado se abre con `tar -xf` como cualquier tar. El proyecto final de este curso lo usa, así que tienes un ejemplo funcionando al que mirar.

Y tres cosas más:

- **Kubernetes** y los orquestadores: quién arranca esa imagen, cuántas copias, y qué hace cuando una se cae. Ahí vuelven el liveness y el readiness del Lab 11.
- **Registries**: dónde se guardan las imágenes y cómo llegan al servidor que las va a correr.
- **CI/CD**: que todo esto lo haga una máquina en cada cambio, en vez de una persona.

6 Para profundizar

- **Cambia una línea de tu código**, vuelve a empaquetar y comprueba con `list-layers` que las capas siguen siendo las mismas cuatro. ¿Cuál de ellas es la única que cambió por dentro?
- **Quita `<layers>`**, vuelve a empaquetar y ejecuta `list-layers`. Compara el resultado.
- **Descomprime el jar** con `unzip -l` y busca tus `application-*.yaml`. Ahí está, en una línea, el motivo por el que no van secretos dentro.
- **Arranca con `--spring.profiles.active=dev` Y con `SPRING_PROFILES_ACTIVE=prod` a la vez.** ¿Cuál gana? ¿Cuadra con la tabla de precedencia?

7 Antes de cerrar

Este lab no deja procesos vivos si paraste los java `-jar` con `Ctrl+C`. Si alguno quedó suelto:

```
lsof -ti:8105 | xargs kill -9
./mvnw clean
```

Lo que te llevas:

La aplicación viaja entera —con su servidor dentro— en un jar por capas que arranca con `java -jar`. La configuración se pone al llegar, no dentro. Y dentro de la caja no van secretos, porque la caja se abre con `unzip`.

Lo que queda pendiente, y abre el Lab 14: todo esto es **una** aplicación. En el Lab 14 la DGT se parte en cuatro procesos con tres bases distintas, y aparecen los problemas que sólo existen cuando el trabajo está repartido.